

Predictive Health Insurance Model

for S.H.I.E.L.D Insurance

by - Deelip Solanki

Objective:

- Develop a high accuracy (>97%) predictive model for Shield Insurance to estimate health insurance premiums based on factors like age, smoking habits, medical history etc.
- The percentage error between actual and predicted values on minimum 95% test data should be less than 10%.
- Create an interactive Streamlit app that an underwriter can use from anywhere.

Project Overview:

We are given past data of premium amounts for around 50k customers.

Using these 12 features below, we have to build a machine learning model to accurately predict the premium amounts for new customers.

```
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                    50000 non-null  int64
1   Gender                                50000 non-null  object
2   Region                                50000 non-null  object
3   Marital_status                        50000 non-null  object
4   Number Of Dependants                  50000 non-null  int64
5   BMI_Category                          50000 non-null  object
6   Smoking_Status                        49989 non-null  object
7   Employment_Status                     49998 non-null  object
8   Income_Level                          49987 non-null  object
9   Income_Lakhs                          50000 non-null  int64
10  Medical History                       50000 non-null  object
11  Insurance_Plan                        50000 non-null  object
12  Annual_Premium_Amount                  50000 non-null  int64
dtypes: int64(4), object(9)
memory usage: 5.0+ MB
```

This project aims to provide healthcare actuaries with a data-driven estimate of healthcare premiums. It can improve risk segmentation, identify important pricing factors, automate large-scale predictions, and provide scenario analysis.

The model will support actuarial decisions while their final judgment remains important.

EDA and Data Cleaning:

1. *standardizing categories:*

The smoking status column had values like 'No Smoking', 'Regular', 'Occasional', nan, 'Smoking=0', 'Does Not Smoke', 'Not Smoking'.

Clearly this column required standardizing some values to one 'No Smoking' category.

2. *Handling Nulls:*

Smoking_Status	11
Employment_Status	2
Income_Level	13

There were some null values in these 3 columns. Although they are very less compared to the size of data we were given and the ideal method to handle them would be to simply drop them, I decided to not loose any data and **fill nulls with some appropriate values**.

For smoking status column:

No Disease	0	1
Smoking_Status		
No Smoking	13814	13574
Occasional	4688	2227
Regular	10312	5374

I found that those who don't have any disease (No Disease = 1) are most likely not a smoker, so I replaced those nulls with 'No Smoking'. Those who has some Disease were most likely a smoker, so I replaced those nulls with 'Regular'.

For Employment status column:

I noticed that, Freelancers, Salaried and Self-Employed had median age of 24, 33 and 42 respectively. So, I used age column to fill the nulls in this column

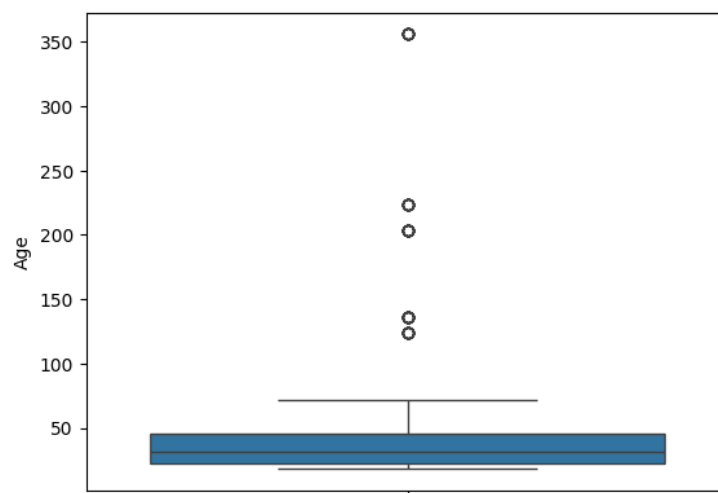
For, Income level:

This was easy to fill, as we just had to use the actual income values from income column to bin them accordingly. Ex: <10, 10-25, 25-40, >40

3. Age, No. of Dependent and Income column seemed to have *outliers*:

Age:

This is the boxplot for Age column.

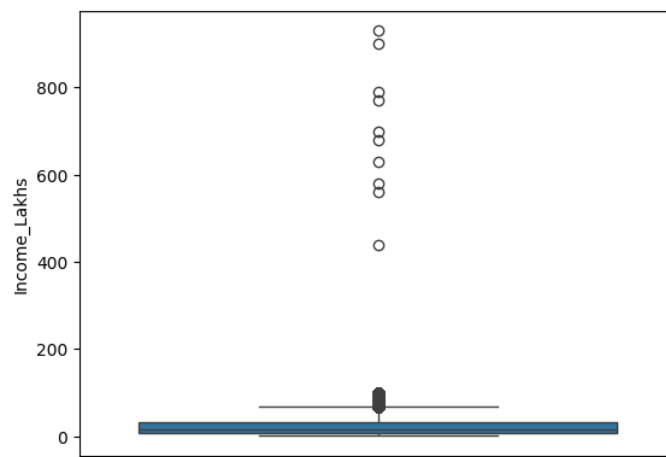


I considered age > 100 to be outliers and replaced them with null and then used the employment status column to fill the nulls with median age of respective group.

No. of Dependent:

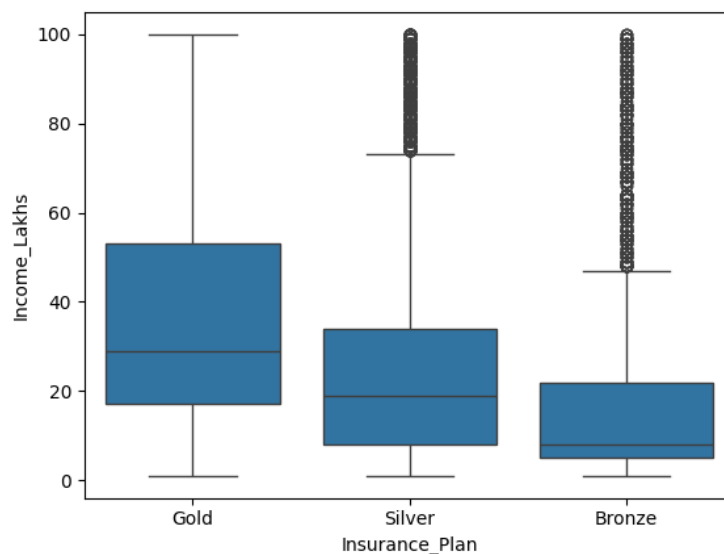
This was the no. of people in a family, supported by that person's income and this had some negative values which were clearly invalid. After talking to the team lead, I got to know that the negative sign was due to **some error in the data collection process**. So, I had to converted them to absolute value.

Income:



Income above 200 lakh was not very impractical, but there were only few points above that level. So, **to make our model robust to outliers**, I decided to replace them with null and fill them based on some other column.

Maybe, the Insurance plan column can help us to treat the income outliers.



People who bought the gold insurance plan generally had a higher median income compared to those who bought the bronze insurance plan.

So, I could use this insurance plan based median income filling for the income outliers.

4. Multiple HOT Coding for medical history:

In the medical history column, I noticed something strange.

Medical History
Diabetes
High blood pressure
No Disease
Diabetes & High blood pressure
Thyroid
Heart disease
High blood pressure & Heart disease
Diabetes & Thyroid
Diabetes & Heart disease

What instantly came into my mind, looking at this, was that I should try to separate the categories that are combined using that ‘&’ symbol, and use multiple columns for each category. By doing this we will have **only 5 distinct groups**:

['Diabetes', 'High blood pressure', 'No Disease', 'Thyroid', 'Heart disease']

Now, our data will have these new columns:

Diabetes	Heart disease	High blood pressure	No Disease	Thyroid
0	0	0	1	0
0	0	0	1	0
0	0	0	1	0
0	0	0	1	0
1	0	1	0	0
0	0	0	1	0
0	0	0	1	0

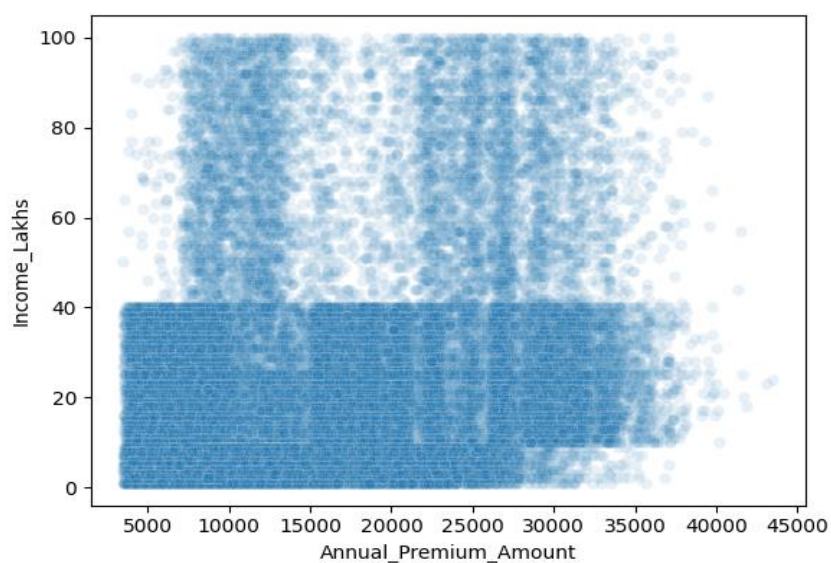
This is not exactly one hot encoding, because we can have multiple ones in a single row because of multiple diseases.

5. Exploring the data for *insights*:

Now, we begin to explore the data with these many columns:

#	Column	Non-Null Count	Dtype
0	Age	10000 non-null	int64
1	Gender	10000 non-null	object
2	Region	10000 non-null	object
3	Marital_status	10000 non-null	object
4	Number Of Dependants	10000 non-null	int64
5	BMI_Category	10000 non-null	object
6	Smoking_Status	10000 non-null	object
7	Employment_Status	10000 non-null	object
8	Income_Level	10000 non-null	object
9	Income_Lakhs	10000 non-null	int64
10	Medical History	10000 non-null	object
11	Insurance_Plan	10000 non-null	object
12	Annual_Premium_Amount	10000 non-null	int64
13	Diabetes	10000 non-null	int64
14	Heart disease	10000 non-null	int64
15	High blood pressure	10000 non-null	int64
16	No Disease	10000 non-null	int64
17	Thyroid	10000 non-null	int64

From the below scatter plot, there is **no clear linear relationship between income and premium amount**.

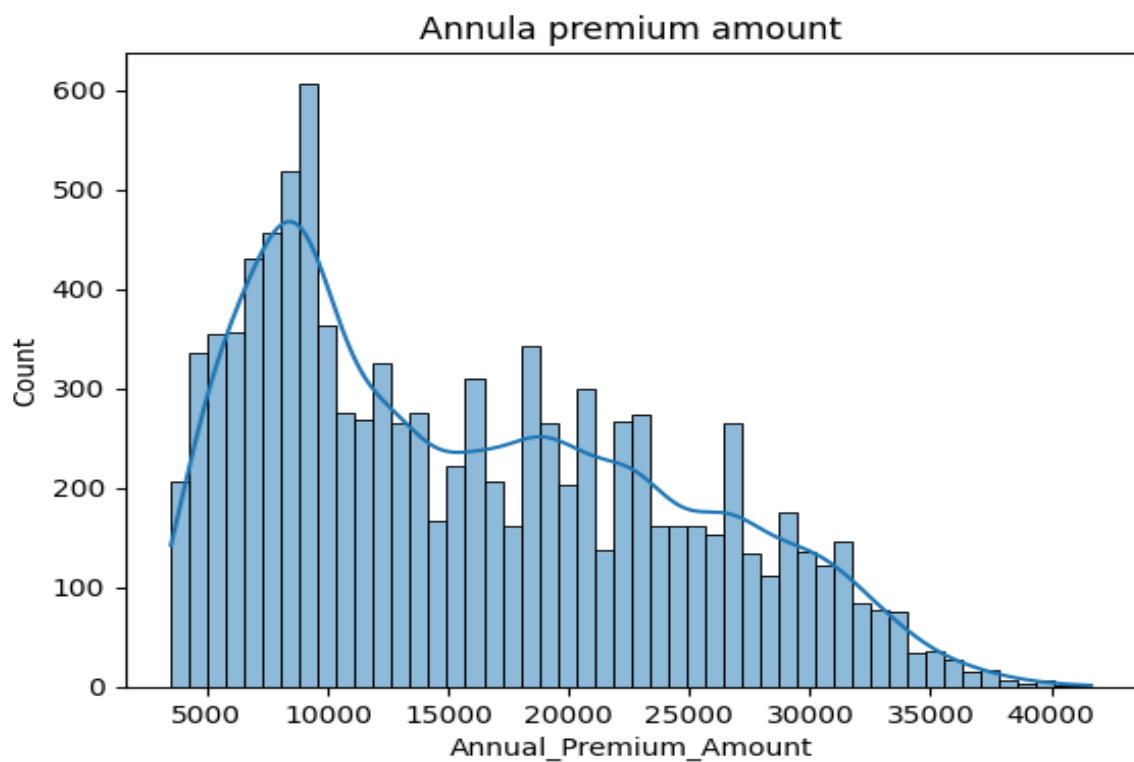


Very first step I did before further EDA, was to get a separate list of categorical and numerical columns. Because the type of chart we use, the type of summary statistics we run is really influenced by the data type, which is either a category or a number.

These two lines of code, does exactly that.

```
cat = list(df.select_dtypes(include=["object"]).columns) #categorical
num = list(df.select_dtypes(exclude=["object"]).columns) #numerical
```

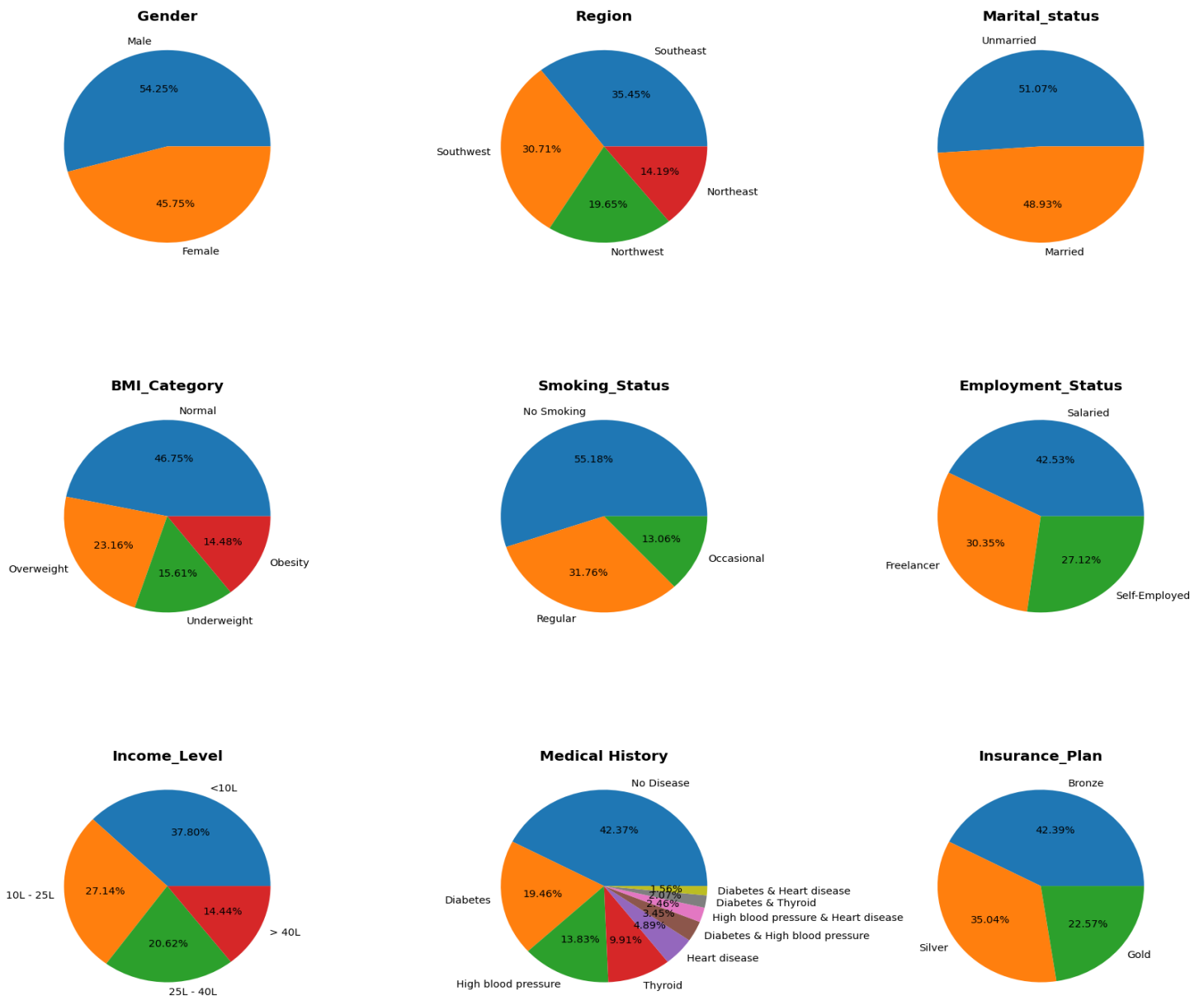
let's look at the **distribution of target variable** using histogram:



This gives an idea about how the values are distributed and its range.

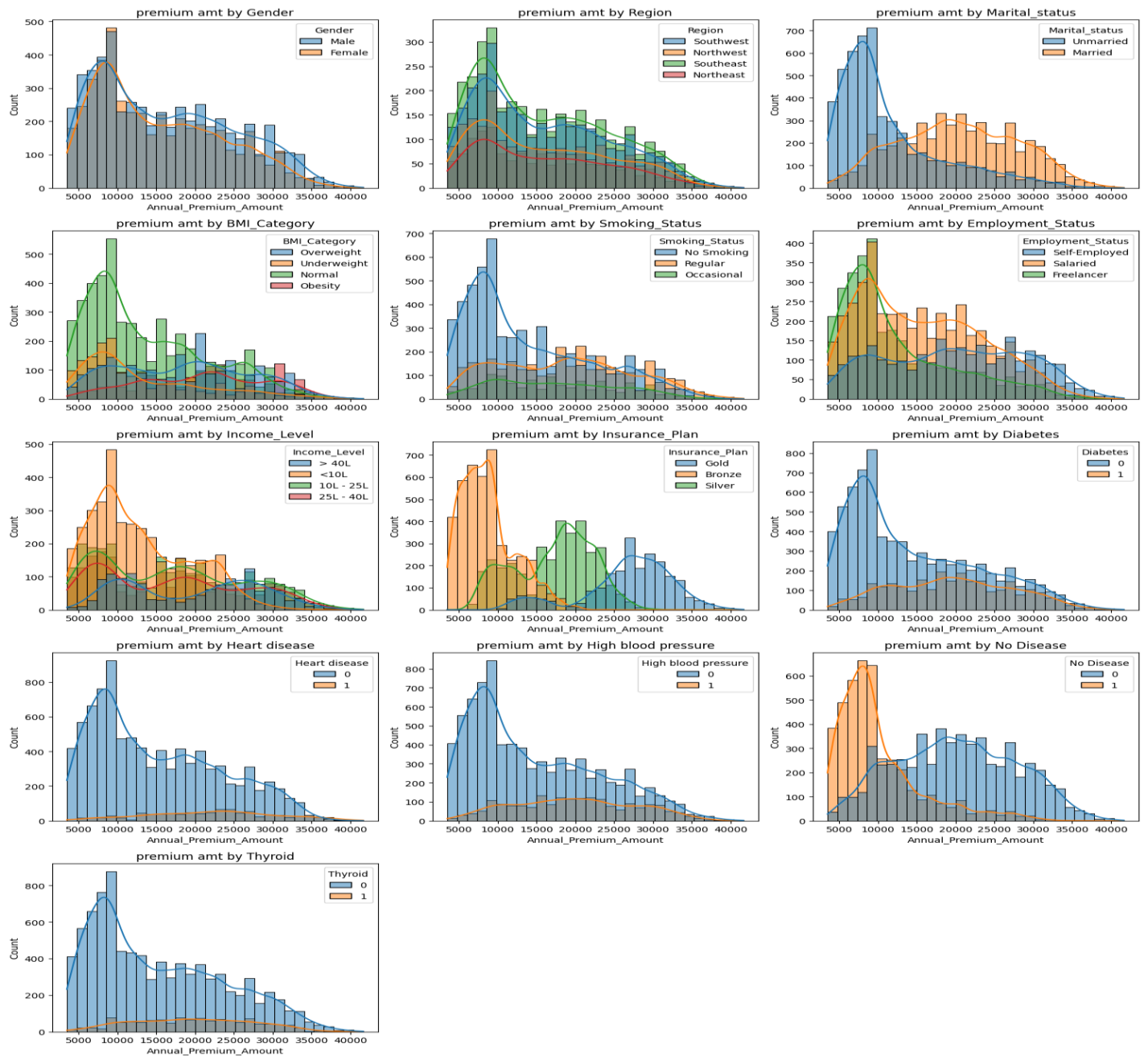
Looking at this we can say that generally premium above about 40,000 would be rare.

A quick view of all **categorical data** in the form of pie chart:



Now what I am interested in knowing after this is, “how my premium amount (that is, the target variable) is distributed **across different categories**”?

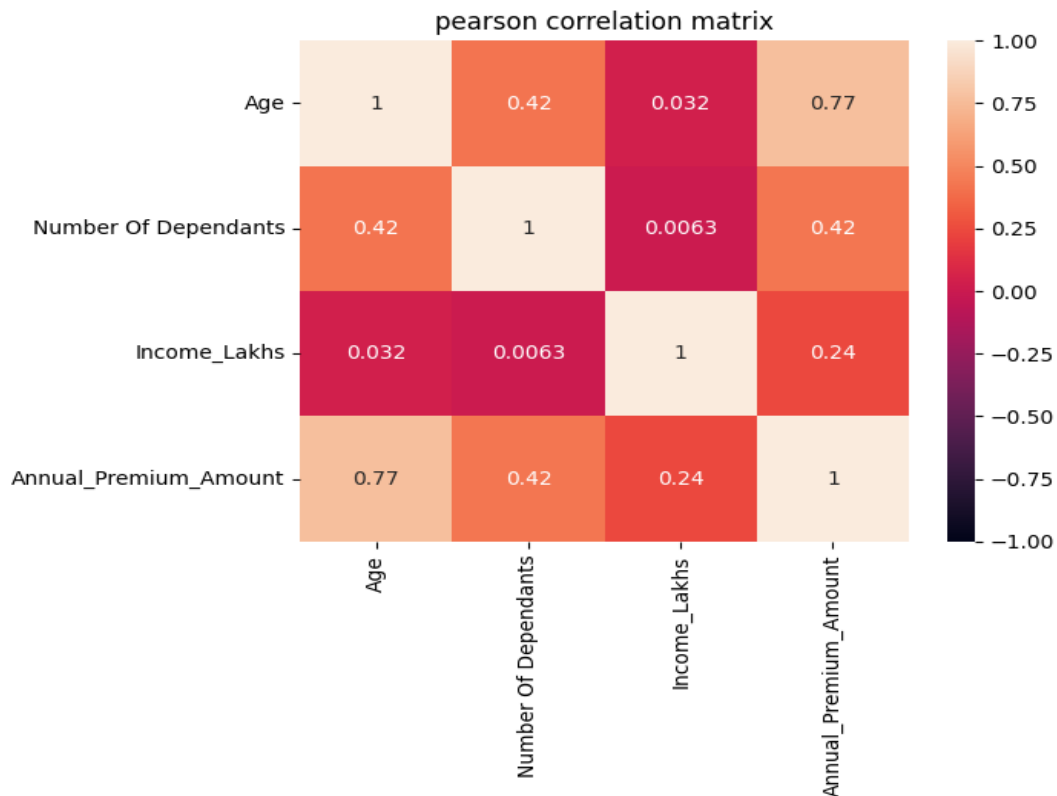
Is it the same as the previous histogram plot that I saw? Or is it different across different categories?



Inference:

- features like **marital status, insurance plan, disease status, BMI** heavily influences **premium price**. (look at the difference in distribution across different categories)
- other features like, smoking status, income level also has some impact on premium price.
- As I expected, **the premium price is less affected by gender and region**. We might drop them during the feature engineering phase, because from this graph they seem to be useless features.

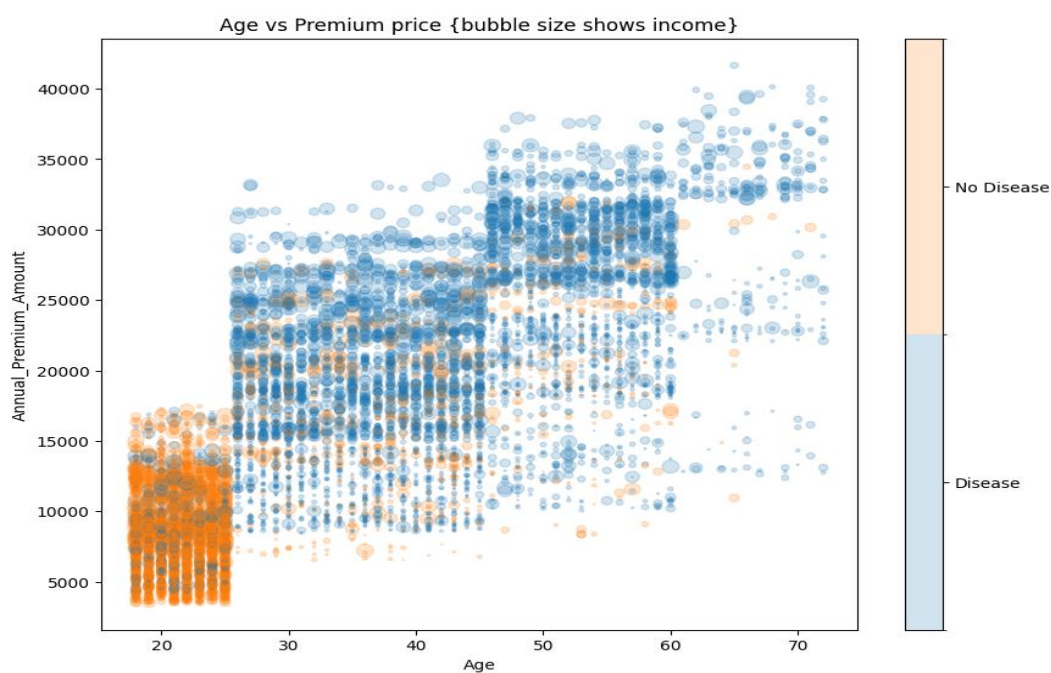
We can use the correlation matrix to check linear **relationship between numerical columns and the target variable**:



Inference:

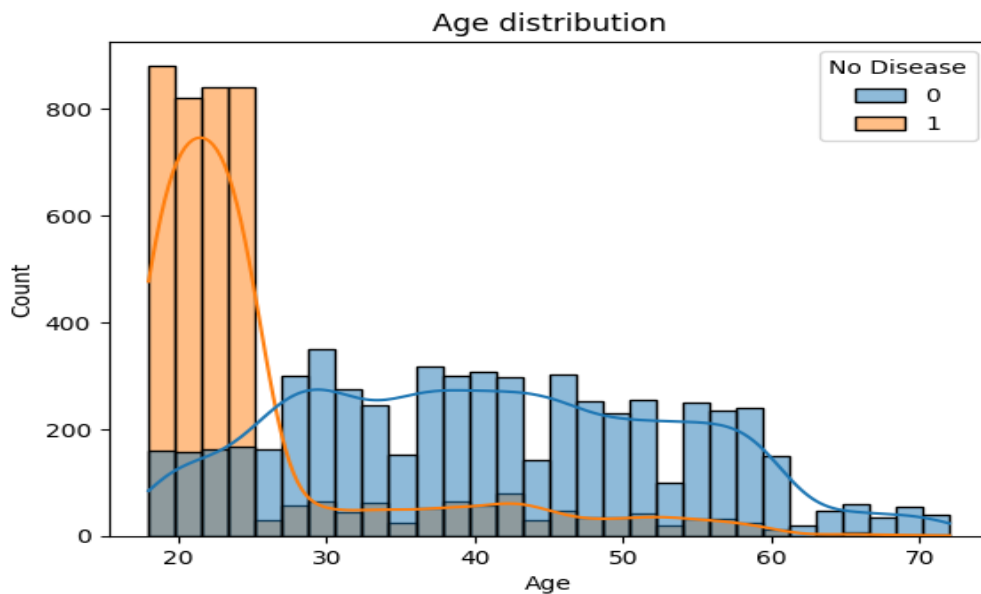
- **target is highly correlated with Age, and slightly with income and NOD**

Because Age is highly correlated with our target, I would like to visualize it as a scatter plot and also with some other variables like income and the disease category.



inference:

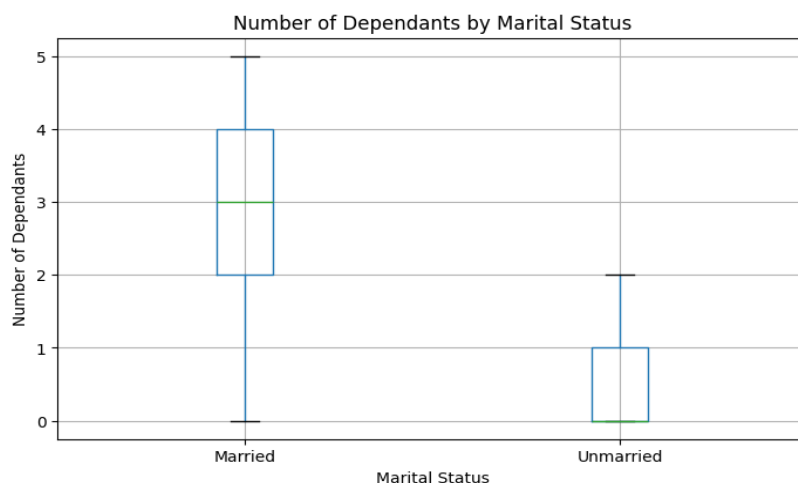
- premium price tends to increase with age.
- high income (large bubble size) people generally have expensive premium plans.
- majority of people who don't have disease are under age of 25, this can also be verified from below graph.



One question that I wanted to find out was **“Why married people had more premium price on average, compared to unmarried people”?**

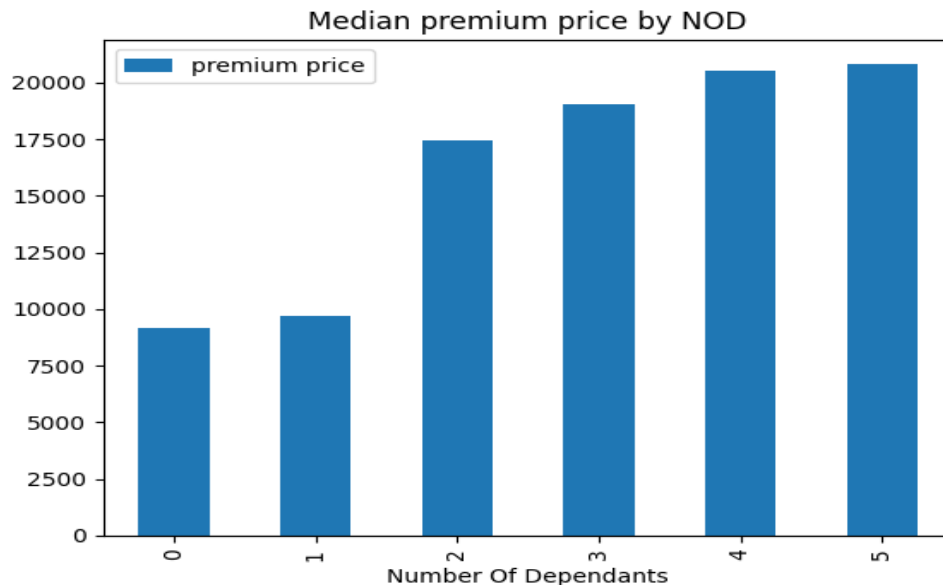
Is it because of No of Dependents? Let's find the median NOD for both groups and see the boxplot.

median NOD	
Marital_status	
Married	3.0
Unmarried	0.0



As expected, **married ones have a greater number of people dependent on their income.**

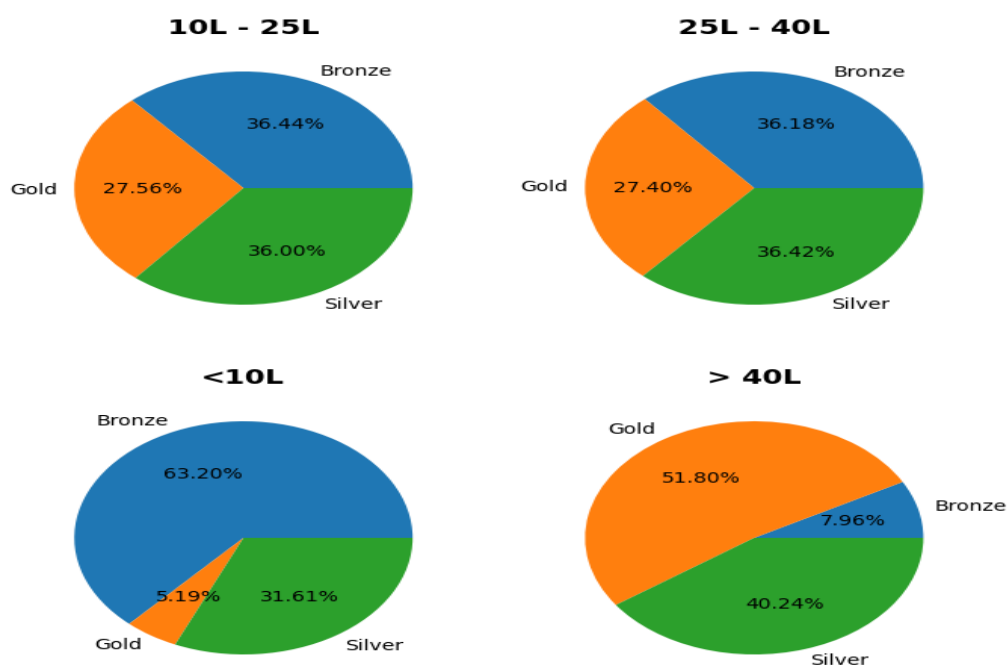
Let's see how this NOD relates with the target variable.



This graph shows, **high NOD will tend to have high price**, which was evident from the correlation matrix also.

We saw before, that the **insurance plan is an important variable for our target.**

Let's see if it has any relation with people's income level. I expect to see high income group to buy more expensive premium plans. In insurance plan we have, gold > silver > bronze, with bronze being the cheapest.



- So, less than 10 Lakhs income group will mostly buy bronze plan and very few of them will go for gold plan. And it is vice versa for greater than 40 Lakhs income group.

Feature Engineering:

- For convenience, I renamed the columns to a standard format and created a new column called 'have_disease' which was basically invert logic for 'no_disease' column, for better readability.
- Then, dropped the 'medical_history' column because we have already created separate columns for each individual disease.
- I also tried creating a new column called 'n_disease', which was basically just the total no of diseases a person has, and drop all the other columns related to diseases.

feature selection:

used sklearn's `r_regression` method, which computes Pearson correlation coefficient 'r' between each feature and the target. It scores the individual effect of each regressor features in predicting target.

For this I had to first convert some categorical features to numerical values.

Here are the features and their **r score (sorted from highest to lowest)**:

```
insurance_plan : 0.834
age : 0.767
n_disease : 0.523
marital_status : 0.517
number_of_dependants : 0.416
bmi_category : 0.333
income_level : 0.269
smoking_status : 0.229
employment_status : 0.146
gender : 0.065
region : 0.001
```

- Insurance plan, Age, no of disease, marital status seems to have strongest influence on target variable.
- NOD, BMI, Income level, Smoking status also has some moderate influence.

Gender and Region have almost no effect on premium price.

If you check back the histogram plots in EDA section, you will notice that categories in gender and region have same distribution of target variable. That means, **knowing someone's gender and region they live in, won't help in predicting the premium amount.**

One more scoring method I will use here is **mutual info regression**, which unlike correlation tests, it **can capture both linear and non-linear relationships.**

```
insurance_plan : 0.616
age : 0.605
n_disease : 0.242
marital_status : 0.169
bmi_category : 0.149
smoking_status : 0.144
number_of_dependants : 0.143
income_level : 0.102
employment_status : 0.064
gender : 0.015
region : 0.0
```

- This also reveals the same insights as r scores.
- Hence, we will **drop gender and region column before building our model.**

Model training:

These are the features that we have before building our machine learning model:

```
Data columns (total 11 columns):
```

#	Column	Non-Null Count	Dtype
0	age	50000 non-null	int64
1	marital_status	50000 non-null	object
2	number_of_dependants	50000 non-null	int64
3	bmi_category	50000 non-null	object
4	smoking_status	50000 non-null	object
5	employment_status	50000 non-null	object
6	income_level	50000 non-null	object
7	income_lakhs	50000 non-null	int64
8	insurance_plan	50000 non-null	object
9	annual_premium_amount	50000 non-null	int64
10	n_disease	50000 non-null	int64

```
dtypes: int64(5), object(6)
```

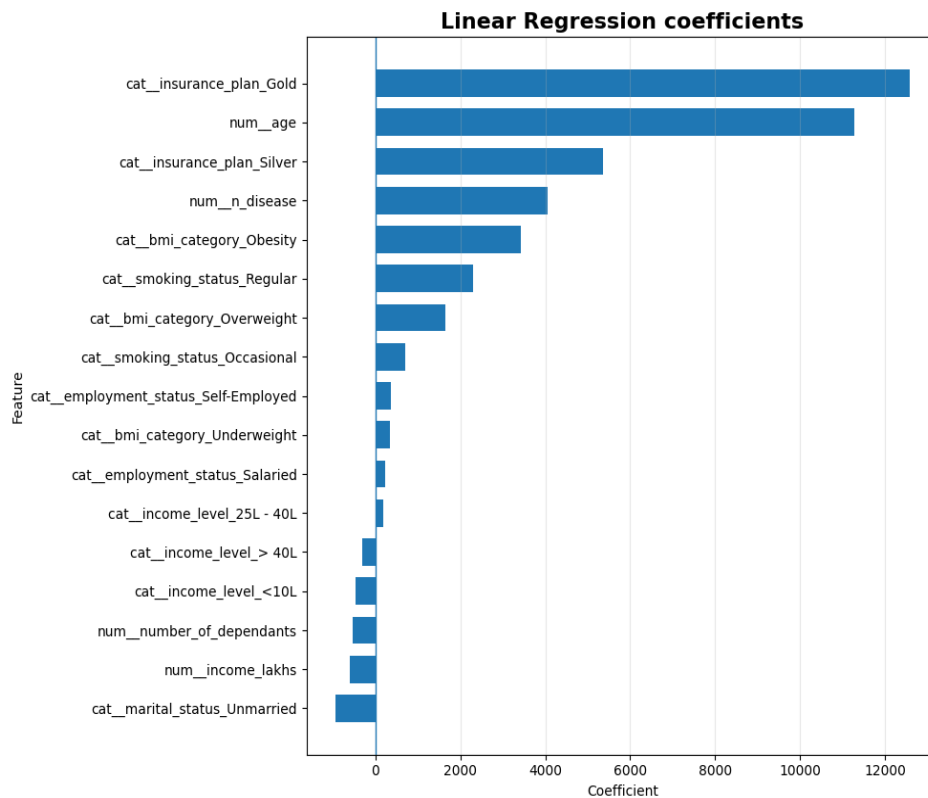
`annual\_premium\_amount` is our target variable and the rest are to be used as predictor variables.

Trying different models:

First, I tried a basic **Linear regression** as the baseline model.

- I created the ColumnTransformer method, in which the numerical columns like age, number_of_dependants, income_lakhs, and n_disease were scaled from 0 to 1 and rest of the columns were one-hot encoded.
- To avoid multicollinearity, I set drop = "first" in one-hot encoding.
- 75% of data was used for training and the remaining 25% were kept for testing the model.
- This model gave an **R2 score of 0.929** on test data.

Now because **linear regression is a highly interpretable model**, I checked the weights for each feature after training.



On the **positive extremes** we have:

- insurance plan (gold, silver)
- age
- number of diseases
- BMI category (obesity, overweight)
- smoking status (regular)

being in these categories with more age and a greater number of diseases can increase the premium amount significantly.

On the **negative extremes** we have:

- marital status (unmarried)
- income lakhs
- number of dependants

One important thing to note in the learned weights was:

Weight for number_of_dependants: **-545.40**

- The interpretation of this value is that, a high number of dependents will tend to have a slightly lower premium amount. But in EDA we concluded that high NOD will tend to have high premium price.
- This is **multicollinearity effect** and indicates that there may be strong correlation between NOD and some other predictor variable.

Previously in EDA we have seen that married people tend to have a higher number of dependents.

Now, converting married -> 1, unmarried -> 0 and calculating correlation with NOD, reveals that there is **a strong correlation between NOD and marital status, with a score of 0.84.**

For a linear regression model, either we will have to combine them into one variable or just use one of them.

After removing marital status:

Weight for number_of_dependants: **610.40**,
and the model performance: R2 -> **0.928**

- Now the coefficient of NOD is giving correct interpretation.

After removing number_of_dependants:

R2 -> **0.929**

- We can **remove NOD**, as we are getting **same performance with just marital status**.

Further, **Using Ridge(L2) or Lasso (L1) regression does not have any noticeable improvement in performance, but L1 can be used for more interpretation on features:**

Interpretation from L1 (Lasso):

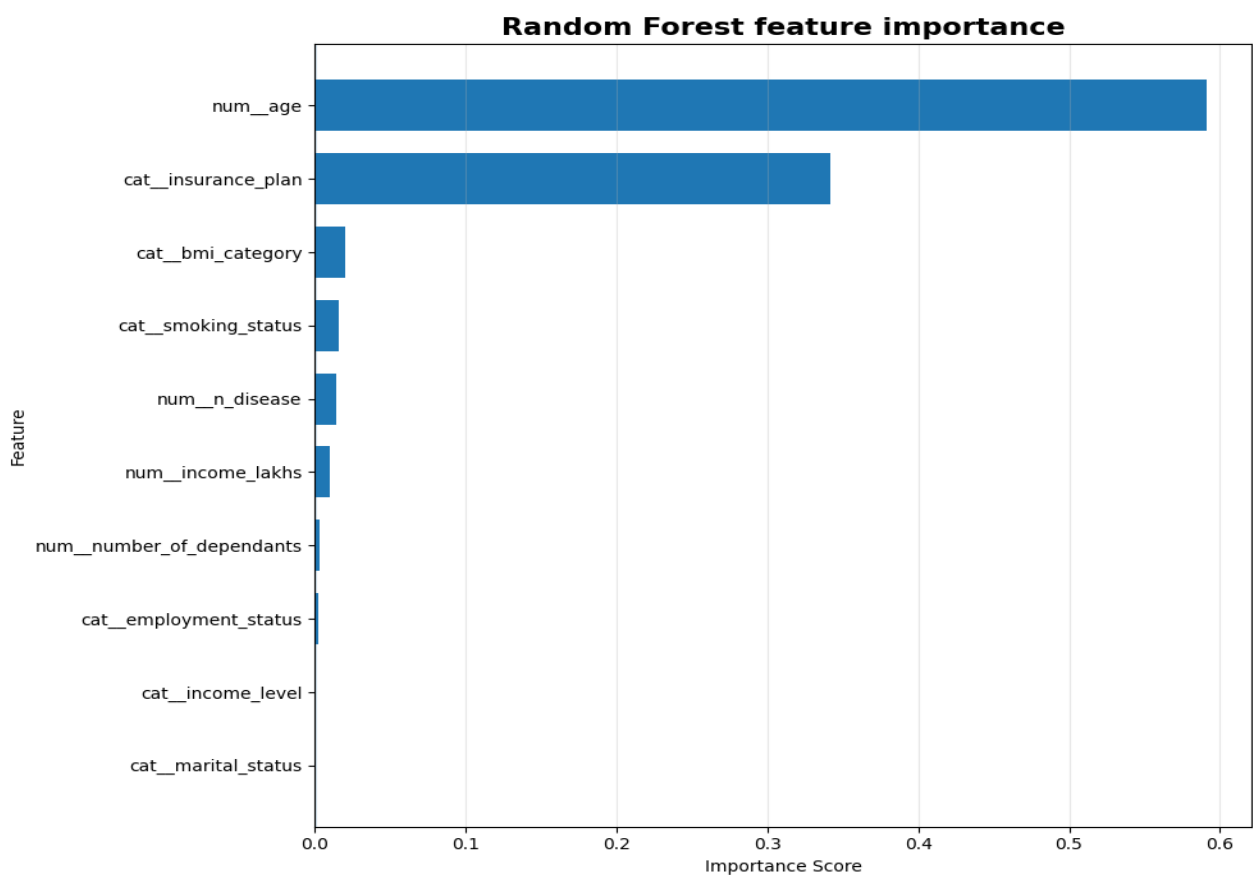
- **L1 regression drives less important features straight to zero**, so the **coefficients of income lakhs and number of dependents are found to be 0.**
- I tried dropping the income lakhs column and the **R2 score** was still the same (**0.929**), so we can **drop the income lakhs column as well.**

After this I tried ordinal encoding instead of One Hot encoding for categorical features, which gave an r2 score of 0.921. which is **less compared to OHE.**

Tree Based Models:

- Starting with **random forest** with the default number of base trees as 100 **gives an r^2 score of around 0.964**.
- In tree-based models, it is generally better to use ordinal encoding over OHE because trees use threshold-based splitting which assumes an order. (but don't use ordinal if categories are nominal)

According to the **Random Forest's impurity-based relative importance** calculation, these are the scores.

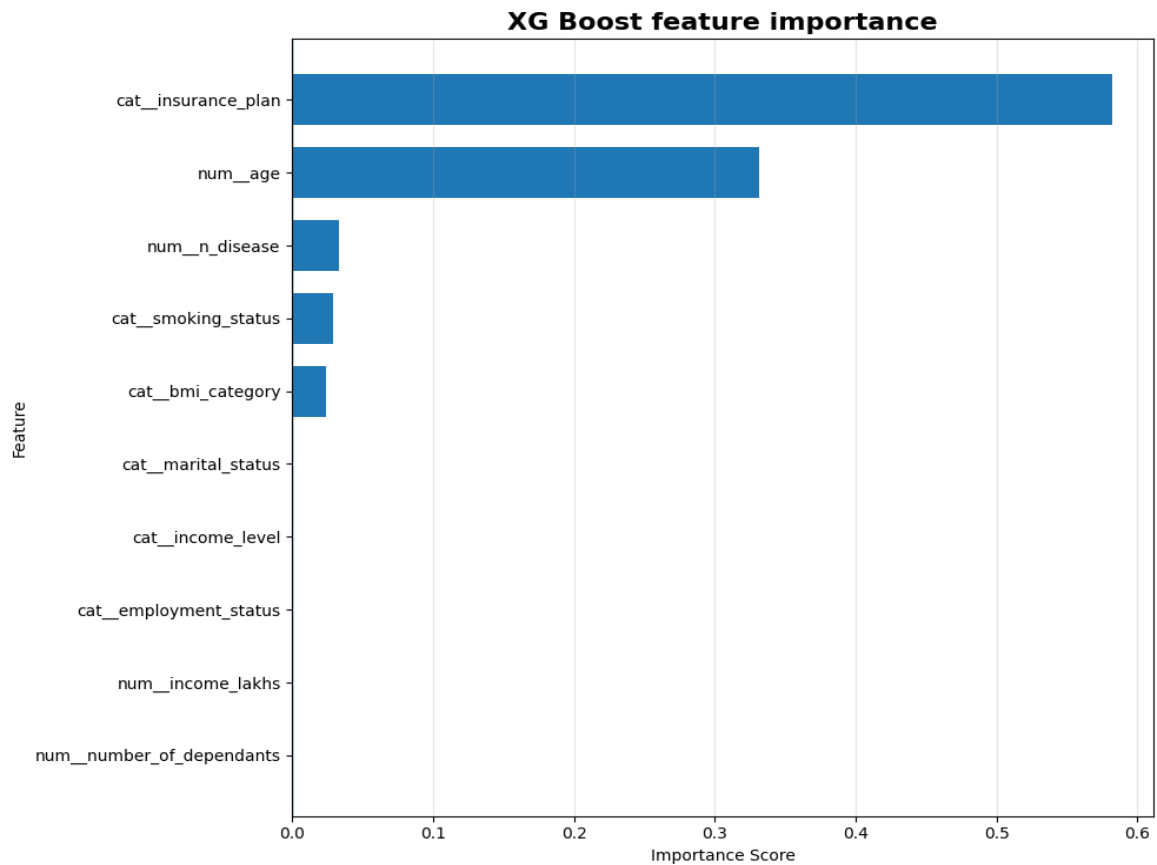


- A high score means, that the feature has contributed more in reducing the impurity while splitting.
- Clearly, **NOD, employment status, income level, and marital status are the least important features according to Random Forest**.
- After training a new model without these features, did not reduce our performance. In fact, it **reduced overfitting on training set from 0.992 to 0.986 while keeping test score same (0.964)**

XGBoost:

Gives an R2 score of **0.972**, which is **better than random forest (0.964)**.

Here is the **feature importance chart** for XGBoost.



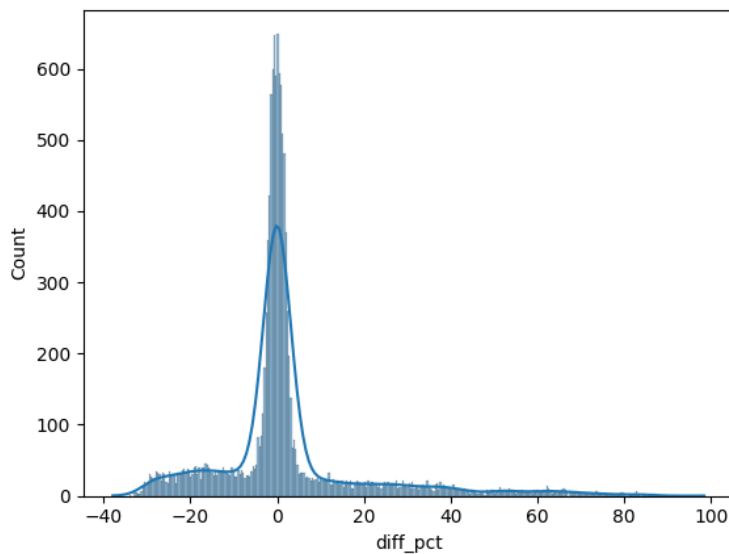
- Very similar to Random Forest.
- **Training another model while keeping only those top 5 features gave same performance.**

Error Analysis:

Till now, our **best model** is XGBoost, with an R2 score of **0.972**.

But now the question is – **Is only high R2 score good enough to deploy a model?**

Comparing the predicted values with their actual values, gave a **distribution of errors** like this:

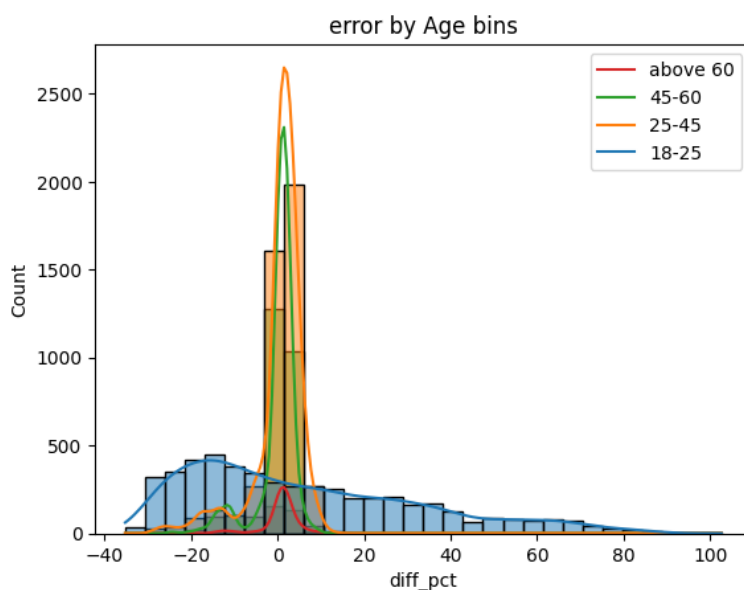


Here, the `diff\_pct` is the percentage difference between the predicted and actual value.

- One of the objectives of this project was that “the percentage error between actual and predicted values on minimum 95% test data should be less than 10%”.
- Analysis on the error data shows that **more than 29% of the data has an error of more than 10% range**. And in some cases, it is so extreme, you can see the distribution going above 40% on the positive side.

We need to find out where this problem is? Is there any specific category for which we have a very strange distribution of errors?

Turns out, for **age group 18-25 we have a very different error plot**. And that is causing the issue.



The data was then divided into two groups:

1. age group 18 to 25 (group 1)
 2. all remaining groups (group 2)
- The group 18-25 was trained on a separate model with more data collected for this group (on genetic risk).
 - Also, I discovered that by using the exact disease information instead of total disease counts, the model performs better (for both groups).
 - **On group 1, we got an R2 score of 0.985 using Random Forest**, with all errors under 10% band.
 - **On group 2, we got an R2 score of 0.999 using Random Forest**, with all errors under 10% band.

This technique is called, **Model Segmentation**, where we split a machine-learning problem into separate groups (segments) and build models differently for each group.

Ready for deployment:

- The model was then deployed to the Streamlit Community Cloud.
- You can view the live app on this link: <https://healthcare-premium-ml.streamlit.app/>
- For code: https://github.com/deelipsolanki-DSML/healthcare_premium_ml



Tech Stack

